Image Deblurring

Using Tikhonov Regularization Across Blur Types

Umay Gokturk, Tiffany Gong, Luna Kim

MATH 441 Mathematical Modelling: Discrete Optimization Problems
University of British Columbia
April 2026

Contents

1 Introduction **3**
1.1 Problem Statement . . . 3

2 Mathematical Background **3**
2.1 Image Representation . . . 3
2.2 The Blurring Model . . . 3
2.2.1 Point Spread Function . . . 3
2.2.2 Noise . . . 5
2.2.3 Boundary Conditions . . . 5
2.2.4 Matrix Computation . . . 5
2.3 Deblurring Methods . . . 6
2.3.1 The Naive Approach for Deblurring the Image . . . 6
2.3.2 Truncated SVD (Singular Value Decomposition) . . . 7
2.4 Tikhonov Regularization . . . 7
2.4.1 The Closed-Form Expression . . . 7
2.4.2 Efficient 2D Implementation via FFT . . . 7
2.4.3 Filter factors . . . 7
2.5 The L-curve . . . 8

3 Linear Motion Blur and Deblurring **9**
3.1 PSF Construction and Blurring . . . 9
3.1.1 Axis-Aligned Motion PSF . . . 9
3.1.2 Angled Linear Motion PSF . . . 9
3.2 Examples with Known PSF . . . 11
3.2.1 Example 2: Angled Linear Motion Deblur . . . 12
3.3 Deblurring a Real Motion Blurred Photo . . . 13
3.3.1 Test 1: Moving Tram . . . 13
3.3.2 Test 2: Camera Shake Toothpaste . . . 14
3.3.3 Test 3: Panning on Moving Object Screenshot with Varying α . . . 15
3.4 Analysis and Discussion . . . 16

4 Color Blurring and Deblurring **17**
4.1 Color Image Representation . . . 17
4.2 PSF and Color Blurring Model . . . 17
4.3 Color Deblurring . . . 17
4.4 Cross-Channel Blurring and Deblurring . . . 18
4.4.1 The Blurring Model . . . 18
4.5 Examples . . . 19
4.5.1 Example 1: Within-Channel Deblur . . . 19
4.5.2 Example 2: Cross-Channel Deblur . . . 20
4.6 Analysis and Discussion . . . 20

5 3D Volumetric Blurring and Deblurring **22**
5.1 The Dataset . . . 22
5.2 3D Blurring . . . 22
5.2.1 3D Gaussian Point Spread Function (PSF) . . . 22
5.2.2 The Convolution Operation for 3D . . . 23
5.2.3 The Fourier Transform for Convolution . . . 23
5.2.4 Adding noise . . . 23
5.2.5 Results . . . 24
5.3 3D Deblurring with Tikhonov Regularization . . . 25
5.3.1 The Tikhonov Regularization Method . . . 25
5.3.2 Implementation Using 3D FFT . . . 25

5.3.3 Choosing α 26
5.3.4 Results 26
5.3.5 Analysis and Discussion 28

6 Conclusion **28**

7 Limitations **29**

8 Future Work **29**

9 Code **29**

1 Introduction

A digital image is composed of pixels, where each pixel holds an intensity value to characterize the color at its position. While these pixels are recorded, the blurring can come from many sources: the air turbulence, the camera's physical limitation (out of focus), object motion, or camera shake. Deblurring defines the process of removing blur from images.

A common but superficial approach to deblurring is sharpening filters, which gives the appearance of deblurring but does not deal with the actual underlying blur. In this project, we explore how deblurring with a linear mathematical model directly addresses the underlying blur.

We treat deblurring as an inverse problem: given a blurred image, we attempt to recover the original image by inverting the blurred operator. Thus, we first must understand how blurring works and how the blurring matrix is constructed before we jump into deblurring images. Then, we will explore how we can use different inverse methods to deblur the blurred images.

1.1 Problem Statement

We use a single Tikhonov-FFT deblurring pipeline to apply it to three different settings with increasing complexity: linear motion blur (where the PSF is a line at an arbitrary angle), color image blur (where RGB channels are coupled through a Kronecker product), and 3D volumetric blur (where the base pipeline extends from pixels to voxels). Naive solution and Truncated SVD are used as comparison baselines to show Tikhonov results.

2 Mathematical Background

2.1 Image Representation

A grayscale image can be represented with a $m \times n$ array, where m and n are the dimension of the image, and each element represents one pixel with a light intensity value. The light intensity value can be either in a full color channel range $[0, 255]$, or a normalized range $[0, 1]$. In this project, we use the normalized range for cleaner representation and computation.
A color image array is similar to the grayscale image, but uses three of $m \times n$ array for each color channel of RGB respectively. Thus, the final representation is as a 3D array, $m \times n \times 3$.

2.2 The Blurring Model

The basic image blurring model as matrix multiplication is the following:

$$Ax = b_{exact} \tag{1}$$

where A is a blurring matrix, x is an original image, and b_{exact} is the exact blurred image.

2.2.1 Point Spread Function

A point spread function (PSF) is a function that describes how blurring spreads from a single bright pixel in the result image. We call the single bright pixel a point source. To obtain the PSF, imagine a single bright pixel in the black background image, and apply blur to spread the point source over its neighboring pixels. Then the resulting image is the PSF [1].
The PSF array can have a smaller dimension than the image, but we use the PSF array with the same size as image with zero paddings for computation.

Gaussian PSF

The most common and well-known blur type is Gaussian blur, which models the blur as a Gaussian distribution, spreading outward from each pixel.

The one-dimensional Gaussian with mean zero and standard deviation σ is

$$g(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{x^2}{2\sigma^2}\right). \tag{2}$$

The parameter σ controls the width of the bell curve. A larger σ means a wider, flatter curve which gives more blur, while a smaller σ means a narrow, peaked curve which means less blur.

For a 2D image, blurring happens in both the horizontal and vertical directions. If the blur is isotropic (blur happens same in all directions) and separable (horizontal and vertical components does not depend on each other),then the 2D Gaussian PSF is the product of two 1D Gaussians [2]:

$$G(x, y) = g(x) \cdot g(y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right). \tag{3}$$

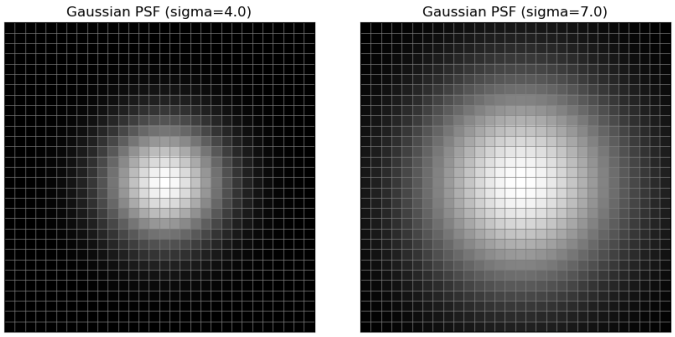


Figure 1: Example Gaussian PSF Visualization

Motion PSF

Another common blur type is motion blur, which occurs when an object or camera moves during image capture. The corresponding PSF models a straight line of pixels along the direction of motion.
The horizontal motion PSF is constructed as follows [1]:

$$p_{ij} = \begin{cases} r^{-1} & \text{if } i = 0 \text{ and } 0 \leq j < r \\ 0 & \text{elsewhere} \end{cases} \tag{4}$$

where i, j are indices, and r is the blur length.

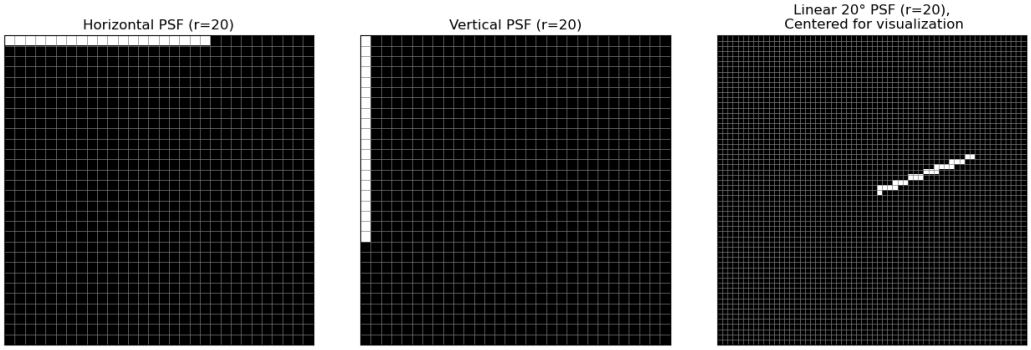


Figure 2: Example Linear Motion PSF (with zero padding) Visualization

2.2.2 Noise

The observed blurred image b can be represented by $b = b_{exact} + e$, where e is an additive noise in the same dimension array as the image. To get a blurred image with noise, the following can be used:

$$b = Ax + e \tag{5}$$

For example, Gaussian white noise for 2D image sized $m \times n$ is $e = \sigma_e \cdot random(m, n)$, where σ_e is a standard deviation.

2.2.3 Boundary Conditions

The blurring matrix A is the combination of PSF and boundary conditions. Boundary conditions are to account for the image edges when the blur is applied to the edge pixels. There are three common boundary conditions: zero, periodic, and reflexive. The zero boundary condition assumes the black image outside of the boundary, and the periodic condition assumes the image is repeated in all directions such as tiles. Lastly, the reflexive boundary condition assumes there are mirrored images outside of the boundary.

This project implicitly choose the periodic boundary conditions as the periodic boundary conditions are the most frequently used for its efficient computation, and zero and reflexive conditions have noticeable drawbacks such as unnatural result or complex computation respectively.

2.2.4 Matrix Computation

When periodic boundary conditions are assumed, the blurring matrix A has a BCCB (Block circulant with circulant blocks) structure as shown in [1].

A BCCB matrix ($MN \times MN$) has the following form:

$$C = \begin{pmatrix} c_0 & c_{M-1} & c_{M-2} & \cdots & c_1 \\ c_1 & c_0 & c_{M-1} & \cdots & c_2 \\ c_2 & c_1 & c_0 & \cdots & c_3 \\ \vdots & \vdots & \vdots & & \vdots \\ c_{M-1} & c_{M-2} & c_{M-3} & \cdots & c_0 \end{pmatrix} \tag{6}$$

where each c_i is an $N \times N$ circulant matrix, while C is an $M \times M$ circulant matrix.

Recall that a square matrix M is normal when $M^*M = MM^*$ for conjugate transpose M^*, and if M is a real matrix, $M^* = M^T$. According to normal matrix definition, any BCCB matrix is normal [1], so A is a normal matrix. Therefore, matrix A can be diagonalized by a unitary matrix such as $A = UDU^*$ where U is a unitary matrix, and D is a diagonal matrix.

For any BCCB matrix, the eigenvectors are the DFT (discrete Fourier transform) basis [1]. By letting $U = F^*$ for the 2D unitary DFT matrix F, and letting $D = \Lambda$ where Λ is a diagonal matrix of eigenvalues of A [1], A can be decomposed as:

$$A = F^*\Lambda F \tag{7}$$

where the columns of F^* are the eigenvectors of A.

Then, DFT can be computed by fast Fourier transform (FFT) efficiently without explicitly constructing F, but implicitly by performing matrix-vector multiplication. This works because a circulant matrix shares the same eigenvectors, so no explicit eigenvalue decomposition is required.

$$b = Ax = F^*\Lambda Fx = F^{-1}\Lambda Fx \tag{8}$$

$F^* = F^{-1}$ because F is a unitary matrix.

```python
def compute_blurred_img(psf, orig_img, noise_amount=0.01):
    """Return the blurred image array with Gaussian white noise by given psf."""

    eigen_val_A = np.fft.fft2(psf)
    blurred = np.real(np.fft.ifft2(eigen_val_A * np.fft.fft2(orig_img)))
    noise = noise_amount * np.random.randn(*orig_img.shape)

    return blurred + noise
```

Python Implementation of Equation (8)

2.3 Deblurring Methods

2.3.1 The Naive Approach for Deblurring the Image

$$X_{naive} = A^{-1}B \tag{9}$$

To recover x, the first instinct would be to invert the blurring model $b = Ax + e$. While this recovers the image exactly in the noiseless case, noise is a natural occurrence. The blurring matrix A is ill-conditioned, meaning some of its singular values are very small or zero. Dividing by these near-zero values amplifies the noise term e dramatically, making the naive solution unusable in practice and motivating the need for regularization. This problem effects 3D in more severe extent because the larger volume contains more high-frequency components that get amplified.

Naive Solution Examples

We applied the naive solution to the blurred images. The naive solutions failed to recover the original images. These failures are the reason filtering methods exist. The rest of Section 2 introduces TSVD and Tikhonov, which differ in how they handle the small eigenvalues that break the naive approach.

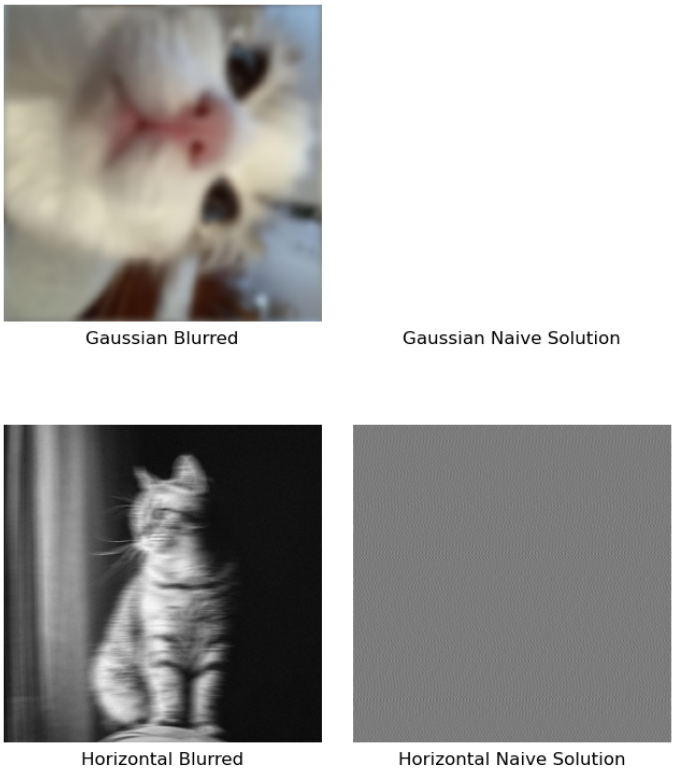


Figure 3: Naive solution results

2.3.2 Truncated SVD (Singular Value Decomposition)

$$A = U\Sigma V^T \tag{10}$$

where U and V are orthogonal matrices, and Σ is a diagonal matrix of singular values. TSVD regularizes the inversion by discarding singular values below a threshold τ, zeroing out their contributions rather than dividing by them. However, this hard cutoff loses useful frequency information entirely and can introduce ringing artifacts near edges, motivating a softer approach to regularization.

2.4 Tikhonov Regularization

Rather than discarding small singular values entirely, Tikhonov regularization damps them smoothly. Instead of enforcing $\|b - Ax\| = 0$ exactly (TSVD), we minimize a weighted sum of the residual and the solution norm:

$$\min_x \left\{ \|b - Ax\|_2^2 + \alpha^2 \|x\|_2^2 \right\}, \tag{11}$$

The first term $\|Ax - b\|^2$ is the residual and measures how well x fits to the observed data. The second term $\alpha^2\|x\|^2$ is the regularization term or solution norm and penalizes solutions that are too large. The parameter $\alpha > 0$ controls the balance between these two terms. Both terms are convex quadratic functions of x, making this a convex optimization problem with a unique global minimum.

2.4.1 The Closed-Form Expression

To solve this Tikhonov problem, we take the derivative with respect to x and set it to zero. Define $f(x) = \|Ax - b\|^2 + \alpha^2\|x\|^2$. This gives us the normal Equation for Tikhonov Regularization[1]:

$$\left(A^T A + \alpha^2 I\right) x = A^T b \tag{12}$$

The matrix $A^T A + \alpha^2 I$ is always invertible when $\alpha > 0$, so we get the unique solution[1]:

$$x_\alpha = \left(A^T A + \alpha^2 I\right)^{-1} A^T b \tag{13}$$

2.4.2 Efficient 2D Implementation via FFT

The Tikhonov solution can be computed very efficiently by working in the frequency domain. Because the blurring matrix A has a BCCB structure, it can be diagonalized using the 2D Fourier transform, which allows us to avoid explicitly forming A.
Substituting $A^T A = F^{-1}|\Lambda|^2 F$ and $A^T b = F^{-1}\bar{\Lambda} F b$ into Equation (13) gives a simplified expression in the Fourier domain:

$$\hat{x}_{\alpha,i} = \frac{\lambda_i}{|\lambda_i|^2 + \alpha^2}\,\hat{b}_i, \qquad i = 1, \ldots, MN, \tag{14}$$

where $\hat{b}_i$ are the Fourier coefficients of the observed blurred image, and λ_i are the eigenvalues of A obtained by taking the 2D DFT of the zero-padded PSF. The restored image is then $\mathbf{x}_\alpha = F^{-1}\hat{\mathbf{x}}_\alpha$. Because (14) is a single element-wise division in frequency space, the entire deblurring step costs $O(MN \log MN)$ rather than the $O((MN)^2)$ of a direct matrix solve.

2.4.3 Filter factors

To find the spectral component of the deblurred images we use filter factors. To find the filter factors we are going to use SVD: $A = U\Sigma V^T$ where $\Sigma = \mathrm{diag}(\sigma_1, \ldots, \sigma_N)$, we substitute into the normal equation solution[1]. Since $A^T A = V\Sigma^2 V^T$ and $A^T = V\Sigma U^T$:

$$x_\alpha = V(\Sigma^2 + \alpha^2 I)^{-1}\Sigma U^T b$$

Since $\Sigma^2 + \alpha^2 I$ is diagonal with entries $\sigma_i^2 + \alpha^2$, and Σ has entries σ_i, the i-th component of this solution is:

$$\frac{\sigma_i}{\sigma_i^2 + \alpha^2} \cdot u_i^T b = \frac{\sigma_i^2}{\sigma_i^2 + \alpha^2} \cdot \frac{u_i^T b}{\sigma_i}$$

So the full solution becomes[1]:

$$x_\alpha = \sum_{i=1}^{N} \phi_i \cdot \frac{u_i^T b}{\sigma_i} \cdot v_i$$

where the Tikhonov filter factors are[1]:

$$\phi_i = \frac{\sigma_i^2}{\sigma_i^2 + \alpha^2}, \qquad i = 1, \ldots, N$$

This factor satisfies $0 < \phi_i \leq 1$ and it gives and insight about frequency of the components:

- When $\sigma_i \gg \alpha$: $\phi_i = \sigma_i^2/(\sigma_i^2 + \alpha^2) \approx 1$. These are the low-frequency components and they are going to be unchanged.
- When $\sigma_i \ll \alpha$: $\phi_i = \sigma_i^2/(\sigma_i^2 + \alpha^2) \approx \sigma_i^2/\alpha^2 \approx 0$. These are the high-frequency components and they are dominated by the noise. So they are going to be suppressed.

The parameter α acts like a threshold. The components that are above α are kept while the below ones are discarded. Higher the α cleaner but blurrier image, lower the α sharper but noisy image.

```python
def compute_tikhonov(psf, blurred_img, alpha):
    """Tikhonov deblurring via 2D FFT using filter factors."""
    eigen_val_A = np.fft.fft2(psf)                          # lambda_i
    b_hat       = np.fft.fft2(blurred_img)                  # b-hat_i
    filter_factor = (np.abs(eigen_val_A)**2) / \
                    (np.abs(eigen_val_A)**2 + alpha**2)     # phi_i
    safe_eigen = np.where(np.abs(eigen_val_A) > 0,
                          eigen_val_A, 1)                   # avoid division by 0
    x_hat = (filter_factor / safe_eigen) * b_hat            # factorized FFT equation
    return np.real(np.fft.ifft2(x_hat))                     # inverse DFT
```

Python Implementation of Equation (14) with Filter Factors

2.5 The L-curve

Selecting the right regularization parameter α is important. If α is too small, the solution overfits the noise, if α is too large, the solution is overly smooth and still blurry. We can use the discrepancy principle, L-curve, to choose the best value.

The L-curve is a log-log plot of the solution norm $\|x\|$ against the residual norm $\|b - Ax\|$ over a range of parameter values. It typically takes an L-shape, hence the name: for small α, the residual is small, but the solution norm is large (noise dominates); for large α, the solution norm is small, but the residual is large (over-smoothing). The optimal parameter lies around the corner of L, where the quantities are balanced.

3 Linear Motion Blur and Deblurring

This section shows the base application of the Tikhonov-FFT pipeline from Section 2 on grayscale 2D images with the added challenge of constructing PSFs at arbitrary angles. If an object moves while a camera's shutter is open, it results in a smeared image of the object, which we call motion blur. The motion blur is one of the most common blur types encountered in photography. In this section, we construct linear motion PSFs for both axis-aligned and arbitrary-angle cases, compare the deblurring results across the naive solution, TSVD, and Tikhonov regularization, and finally attempt to deblur real world motion blurred photographs by estimating the PSF parameters.

3.1 PSF Construction and Blurring

3.1.1 Axis-Aligned Motion PSF

The horizontal linear motion PSF was introduced in Section 2.2.1. For a blur length r, the PSF places $1/r$ along the r pixels in the horizontal direction, and zero elsewhere. The vertical linear motion PSF has almost the same structure except placing the $1/r$ along the vertical direction. As mentioned earlier, the PSF array is padded with zeros to the same size as the given image size for FFT computation. Below are the blurred results on a grayscale test image with the length $r = 20$ and Gaussian noise of standard deviation $\sigma_e = 0.01$.

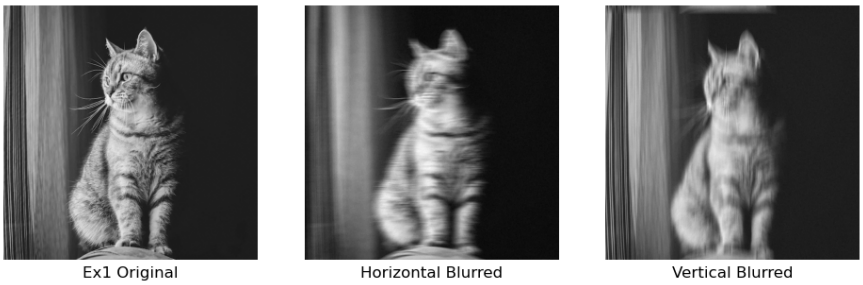


Figure 4: Grayscale test image and its axis-aligned motion blurred images ($r = 20$, $\sigma_e = 0.01$)[1]

3.1.2 Angled Linear Motion PSF

Real motion blur rarely aligns with the image axes. To construct the PSF for an arbitrary angle θ, we trace a line of length r at angle θ and collect a set of pixel positions along it. Then, at each pixel position, we place $1/R$, where R is the number of pixels in the set.
In the process of collecting a set of pixel positions, we round to the nearest integer to specify a pixel. Each pixel position (p_i, p_j) at a sampled distance $t_k \in [0, r-1]$ is calculated as follows:

$$(p_i, p_j) = (\text{round}(\cos\theta \cdot t_k),\ \text{round}(\sin\theta \cdot t_k)), \quad t_k \in [0, r-1] \tag{15}$$

However, we found problems while exploring and testing images with various PSFs. With periodic boundary condition and the FFT-based computation, the index position $(0, 0)$ of the PSF array is treated as a reference point where the point source lands in the output. If non-zero PSF values are placed far from the position $(0, 0)$, the blurred image ends up shifted, wrapping around the boundaries, which is somehow expected due to our boundary condition choice.

On top of that, since the angled PSF construction only produces positive pixel coordinates, the blur direction does not always match the intended angle. For example, the given angle meant to go up-right direction would actually be blurred down-right. To avoid both issues, we wanted to make sure the PSF trace always starts at $(0, 0)$. Therefore, we decided to flip the image before applying blur based on the given angle's quadrant

[1] Original image by Arturo Anez from Pixabay `https://pixabay.com/photos/cat-pet-animal-nature-domestic-cat-9569386/`

so that the trace is correctly anchored at the top-left corner and the blur direction is as intended. After blurring is computed, we flip the image back.

```python
def get_pixel_pos_for_angled_linear_motion(length, angle_deg):
    """Return a list of pixel coordinates for a motion blur line of given length and angle.
    All the position is shifted to positive axis to align it for psf."""

    len_list = np.linspace(0, length - 1, num=(length * 2))
    pos_set = set()
    shifted_rad = np.radians(angle_deg) + (np.pi / 2)

    for l in len_list:
        x_pos = abs(round( np.cos(shifted_rad) * l ))
        y_pos = abs(round( np.sin(shifted_rad) * l ))
        pos_set.add((x_pos, y_pos))

    return list(pos_set)

def build_linear_motion_psf(length, angle_deg, img_shape):
    """Build angled linear motion psf with padding zeros."""

    psf = np.zeros(img_shape)
    positions = get_pixel_pos_for_angled_linear_motion(length, angle_deg)
    pos_count = len(positions)
    for pos in positions:
        psf[*pos] = 1 / pos_count

    return psf
```

Python Implementation of Equation (15)

Note that we used $\pi/2$ shifting when computing pixel coordinates in our implementation. This is because in array indexing, the origin $(0, 0)$ is at the top-left, while in standard math convention, the angle θ is measured from the positive x-axis anti-clockwise in a bottom-left origin system.

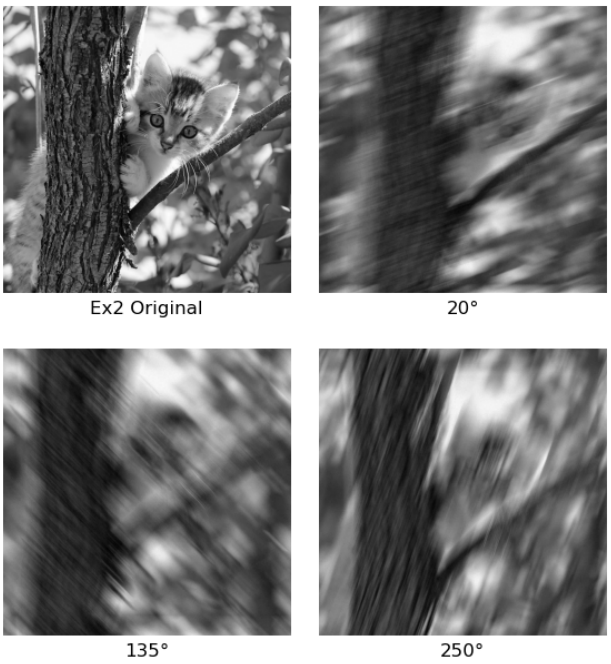


Figure 5: Grayscale test image and its angled linear motion blurred images ($r = 50$, $\sigma_e = 0.01$)[2]

For each θ value, we can see the smudge direction is aligned with the θ direction.

[2]Original image by Pavel from Pixabay `https://pixabay.com/photos/cat-kitten-tree-curious-tabby-1647775/`

3.2 Examples with Known PSF

When the PSF is known, we can directly use the PSF to construct deblurring filters. In this section, we explore Tikhonov regularization in depth. All examples are applied to grayscale images with Gaussian noise of $\sigma_e = 0.01$.

We apply TSVD and Tikhonov regularization on horizontally motion blurred image, both methods filter out the small eigenvalues to suppress noise blow-up. The tolerance for TSVD and the regularization parameter α for Tikhonov were selected using L-curve analysis, and also along with some visual judgement.

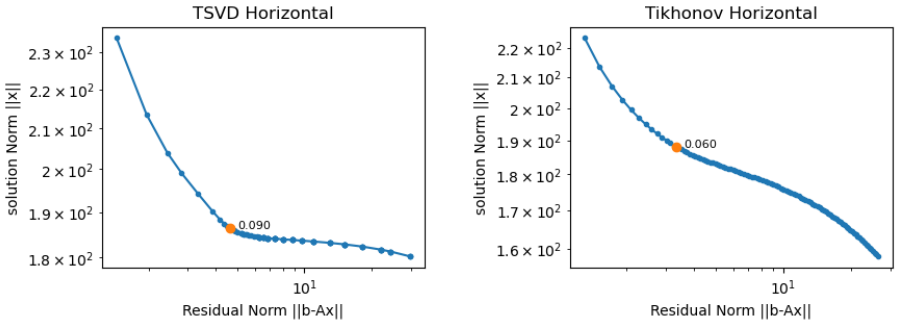


Figure 6: TSVD and Tikhonov L-curve for horizontally motion blurred image

From the L-curves in Figure 6, we selected a tolerance of 0.09 for TSVD and $\alpha = 0.06$ for Tikhonov. With these parameters, both TSVD and Tikhonov successfully deblur the blurred image with minimal visible noise and softening, as shown in Figure 7. Compared to the complete failure of the naive solution, the filtered methods clearly show the benefit of suppressing small eigenvalues. The whiskers and fur details of the cat are recovered well in both methods, but clearly the result with Tikhonov regularization has fewer ringing artifacts and shows a smoother image. We only show the horizontal case here. The vertical case is available in the GitHub repository (see Section 9).

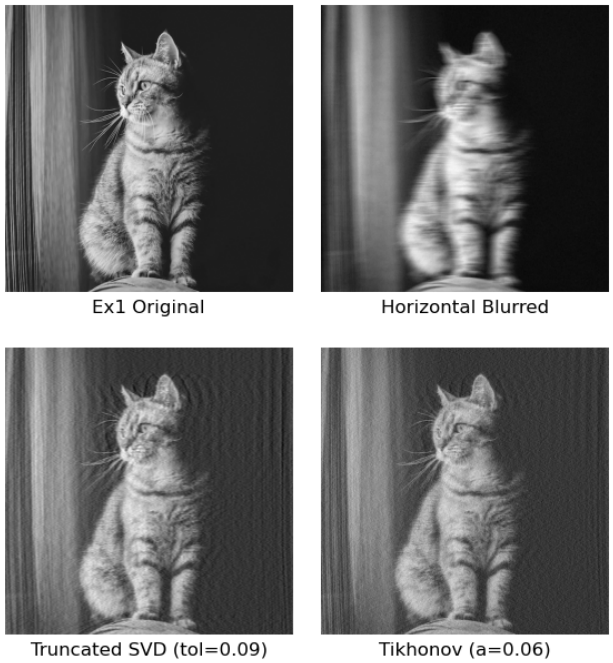


Figure 7: TSVD and Tikhonov results on horizontally motion blurred image

3.2.1 Example 2: Angled Linear Motion Deblur

We applied angled linear motion blur with $r = 50$ at three angles $\theta \in \{20, 135, 250\}$ in Section 3.1.2. In this section, we apply TSVD and Tikhonov regularization on the $\theta = 20$ motion blurred image.

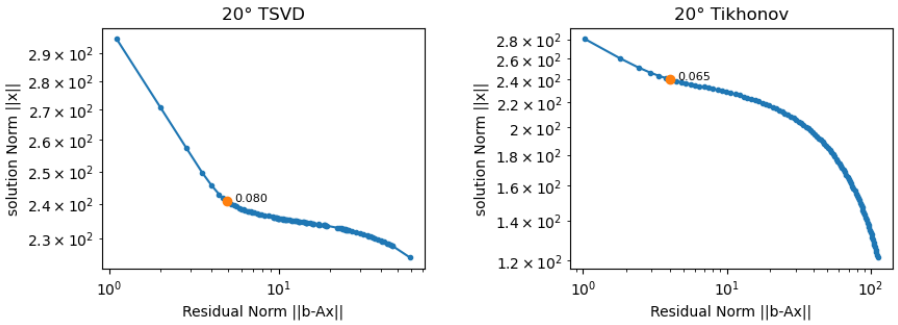


Figure 8: TSVD and Tikhonov L-curve for 20° angled linear motion blurred image

Same as before for the horizontal motion deblur, we first check the L-curves for both methods to the 20 blurred image, as shown in Figure 8. From the L-curves, we selected a tolerance of 0.08 for TSVD and $\alpha = 0.065$ for Tikhonov.

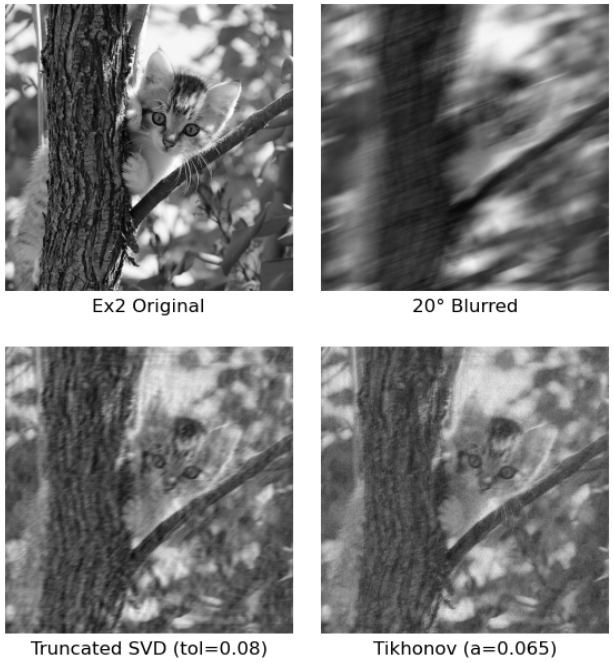


Figure 9: TSVD and Tikhonov results on 20° angled linear motion blurred image

Similar to Example 1, both methods successfully recover the original image well in Figure 9, confirming that the angled PSF construction and the flip-based alignment trick work as intended. The same behavior was observed for the 135° and 250° cases, which are available in the GitHub repository.

Interestingly, the same tolerance and α values worked well for all three angles $\theta \in \{20, 135, 250\}$. This is because we used the same blur length $r = 50$, and rotating the PSF does not significantly change the magnitudes of the blurring matrix's eigenvalues, but only their positions in the frequency domain. Small rounding errors from rotation may occur, but the effect seems negligible in practice. Since the noise level

(standard deviation σ_e) is also the same, the L-curves, which plot solution norm against residual norm, are effectively identical across angles.

3.3 Deblurring a Real Motion Blurred Photo

In the previous examples, the PSF was known exactly because we constructed the blur ourselves. However, for real world photographs, the PSF is unknown and must be estimated. In this section, we attempt to deblur three real motion blurred photographs by estimating the PSF parameters (length r and angle θ) visually, and then applying Tikhonov regularization.

The linear motion blur PSF is determined by two parameters: the blur length r and the angle θ. We estimate these by inspecting the visible direction of smearing and approximately measuring the length of smearing. We also tune the regularization parameter α manually along with small adjustments to r and θ until the result looks visually reasonable.

Note that the test examples shown here are the result after fine-tuning with many experimental attempts. Additionally, all original photos are converted to grayscale images using the Pillow library for the tests since common photo editors tend to use full RGB color range even for black and white images. For the methods introduced in this project, the image ratio does not matter, but most of the following examples use 1:1 ratio only for a cleaner layout of figures.

3.3.1 Test 1: Moving Tram

The first test is a photograph of a tram in motion. The final attempt parameters are $r = 27$, $\theta = 1$, $\alpha = 0.112$.

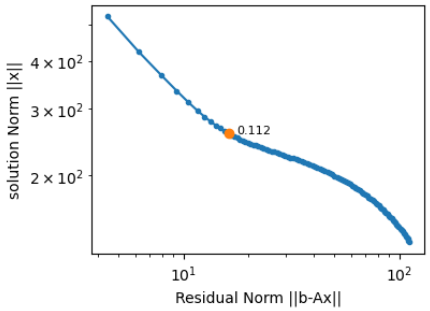


Figure 10: Tikhonov L-curve for the tram image



Figure 11: Tikhonov results on the tram image[3]

The estimated angle is nearly horizontal, consistent with the tram moving along the tracks. The deblurred result recovers sharper edges on the tram, and the overall result image is softened significantly. Because the motion blur in the original photo only affects the tram, not the static background buildings, applying a uniform deblurring filter resulted in ringing artifacts and warping in the building regions in the image. This is a limitation of the linear model. It assumes the entire image was blurred by the same PSF, which happens rarely in real photographs where regions can have different blur angles or types.

3.3.2 Test 2: Camera Shake Toothpaste

The second test is a photograph of toothpaste. The photo is taken by moving the camera fast in a linear motion. The final attempt parameters are $r = 26,\ \theta = 315,\ \alpha = 0.048$.

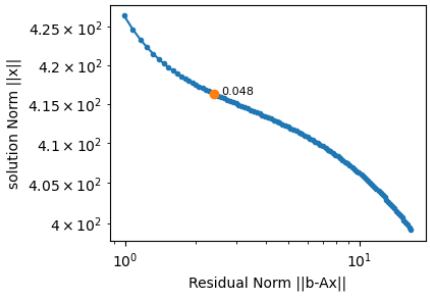


Figure 12: Tikhonov L-curve for the toothpaste image



Figure 13: Tikhonov results on the toothpaste image (Photo taken by the author)

Camera shake effects apply amost uniformly on the entire scene, which fits the linear blur model better than Test 1. As a result, deblurred image shows improved text readability and overall sharpness of toothpaste

[3]Original image by Marco Hofmann from Pixabay `https://pixabay.com/photos/tram-black-and-white-city-5451176/`

shape across the entire image. However, our estimation is just an approximation, and shake motion is not a perfectly straight line, which explains why some ringing artifacts remain.

3.3.3 Test 3: Panning on Moving Object Screenshot with Varying α

The third test is a screenshot from a photography tutorial video demonstrating camera panning. The final attempt parameters are $r = 115,\ \theta = -1,\ \alpha \in \{0.085, 0.145, 0.210\}$.

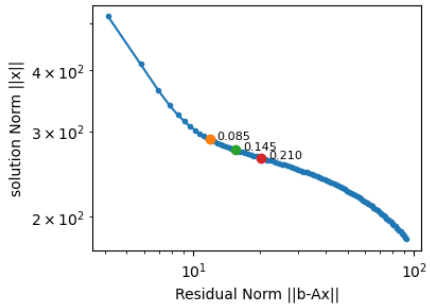


Figure 14: Tikhonov L-curve for the screenshot

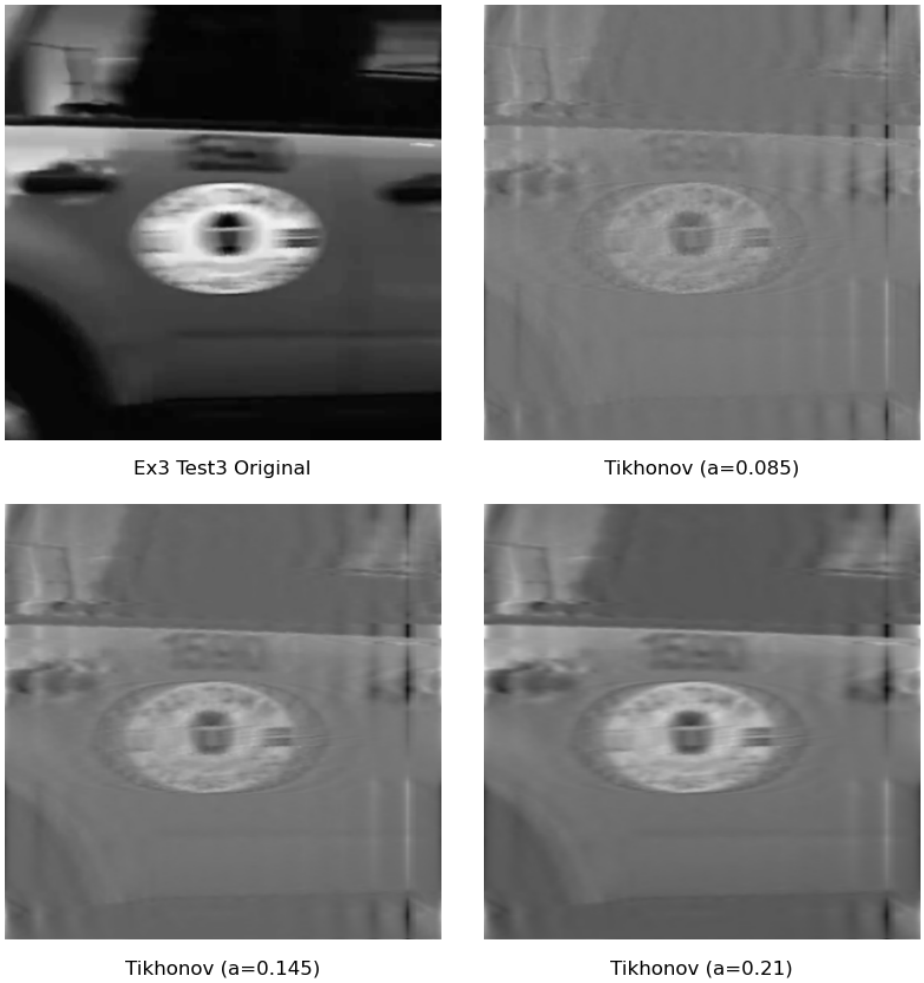


Figure 15: Tikhonov results on the screenshot[4]

The panning screenshot is a challenging case because of the large blur length $r = 115$ and also both the camera and the object are moving with different speed, which can make the problem highly ill-conditioned. This is why the choice of α has a visible impact compared to the earlier two tests.

The result shows that the relatively smaller α at 0.085 preserves more detail but amplifies noise, while large α at 0.210 produces a smoother result at the cost of sharpness. The middle α at 0.145 balances the two.

3.4 Analysis and Discussion

Throughout all the testing with the examples, we observed clear differences among the three deblurring methods. We also found Tikhonov regularization works well when it is applied to real world motion blur.

The naive solution failed due to noise amplification. Since it simply inverts the blurring model, small eigenvalues close to zero lead to results where any noise in the blurred image takes over the entire result.
TSVD and Tikhonov regularization both successfully deblur the blurred image when the PSF was known exactly. TSVD ignores the small eigenvalues less than a given threshold, and Tikhonov damps them smoothly balancing the residual and the solution norm with the optimal α value.
In our experiments, both methods produced visually comparable deblurred results for known PSF cases, although Tikhonov tends to be more stable and smooth overall since it does not introduce a sharp cutoff like TSVD.

For the real photo tests with estimated PSFs, the results were more limited. Even after fine-tuning r, θ, and α parameters, the deblurred outputs were not as clean as in the synthetic cases. This is most likely because the real PSF is rarely a perfectly straight line with a uniform intensity. Camera shake, panning, object moving, or a combination of these produce more complex paths that our linear motion model cannot fully capture. In addition, regions of the image that were not blurred (such as the static buildings in the Test 1 Tram Photo) end up being warped by the deblurring filter, since the filter was applied to the entire image uniformly.

The L-curve was a useful tool for selecting the optimal α throughout the overall process. However, we also found that using the L-curve as a guide to identify a potential optimal range, and then applying visual judgment based on the expected result can produce a more satisfied deblurred image.

Overall, our experiments confirm that Tikhonov regularization is an effective deblurring method, especially when the PSF is known accurately, but its performance heavily depends on how accurately the PSF matches the true blur. This sensitivity to PSF accuracy is the biggest limitation for applying these methods to real world images.

The same Tikhonov-FFT pipeline is reused in the next two sections, with channel coupling added in Section 4 and a third dimension added in Section 5.

[4]Screenshot from "Photographing Motion with Camera Panning" by Mango Street, YouTube. `https://youtu.be/rY-Dc3VnkT8?t=34`

4 Color Blurring and Deblurring

Color image deblurring extends the grayscale image deblurring by accounting for the multi-channel structure of colored images. We apply the spectral filtering methods independently to each RGB channel.

4.1 Color Image Representation

A digital color image is represented as a three-dimensional array X where the three two-dimensional arrays $X(:,:,1)$, $X(:,:,2)$, and $X(:,:,3)$ correspond to the red, green, and blue (RGB) intensity channels respectively. Each channel is treated as an individual grayscale image, and together they form the full color of an image.

In practice, color images are recorded using a charge-coupled device (CCD) sensor with a Bayer pattern filter, which assigns different color sensitivities to adjacent pixels. If this pattern is misaligned, light from one channel can bleed into another, known as cross-channel blurring, explored further in Section 4.4.

4.2 PSF and Color Blurring Model

Within-channel blurring uses the same 2D Gaussian PSF and FFT-based approach from Section 2, applied independently to each RGB channel. We use Gaussian noise with standard deviation $\sigma_e = 0.02$.

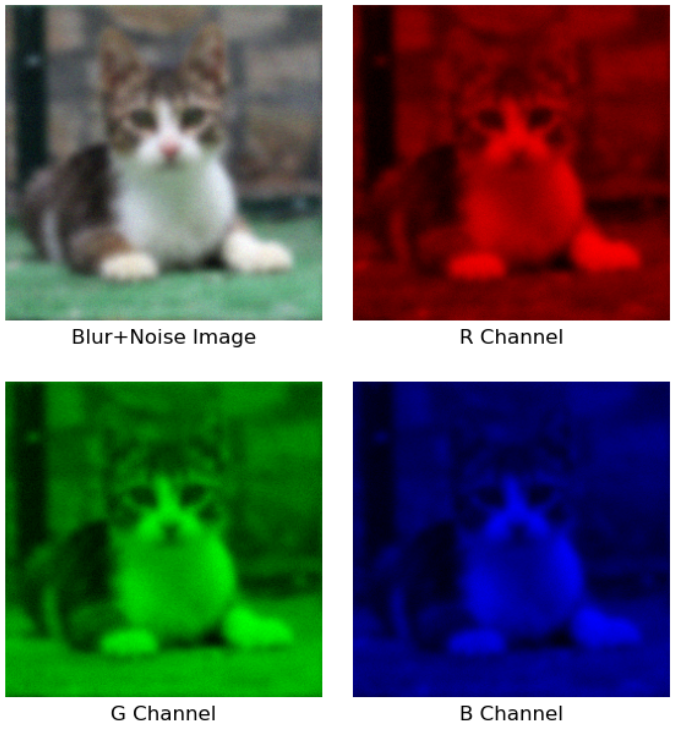


Figure 16: Blurred and noisy image split into its red, green, and blue channels.

4.3 Color Deblurring

When only within-channel blurring is present, the naive solution, TSVD, and Tikhonov from Section 2 are each applied independently to the three RGB channels using the same parameter across all channels to preserve color balance. For Tikhonov, we use the L-curve to select α by aggregating the solution and residual norms across all three channels into a single combined curve.

4.4 Cross-Channel Blurring and Deblurring

When cross-channel blurring is present, it is modelled by a 3×3 color mixing matrix:

$$A_{\text{color}} = \begin{bmatrix} a_{rr} & a_{rg} & a_{rb} \\ a_{gr} & a_{gg} & a_{gb} \\ a_{br} & a_{bg} & a_{bb} \end{bmatrix}, \tag{16}$$

where a_{ij} represents how much of channel j bleeds into channel i, and each row sums to one. The full blurring model is then:

$$(A_{\text{color}} \otimes A)\, x = b, \tag{17}$$

When $A_{\text{color}} \neq I_3$, the three channels are not independent, and we encounter cross-channel blurring. Blurring is applied in two steps: firstly, the within-channel PSF blur, then cross-channel mixing.

4.4.1 The Blurring Model

To understand $(A_{\text{color}} \otimes A)\,\mathbf{x} = \mathbf{b}$, recall that the Kronecker product, $P \otimes Q$ of an $m \times m$ matrix P and an $n \times n$ matrix Q is the $mn \times mn$ block matrix:

$$P \otimes Q = \begin{pmatrix} p_{11}Q & p_{12}Q & \cdots & p_{1m}Q \\ p_{21}Q & p_{22}Q & \cdots & p_{2m}Q \\ \vdots & \vdots & \ddots & \vdots \\ p_{m1}Q & p_{m2}Q & \cdots & p_{mm}Q \end{pmatrix}.$$

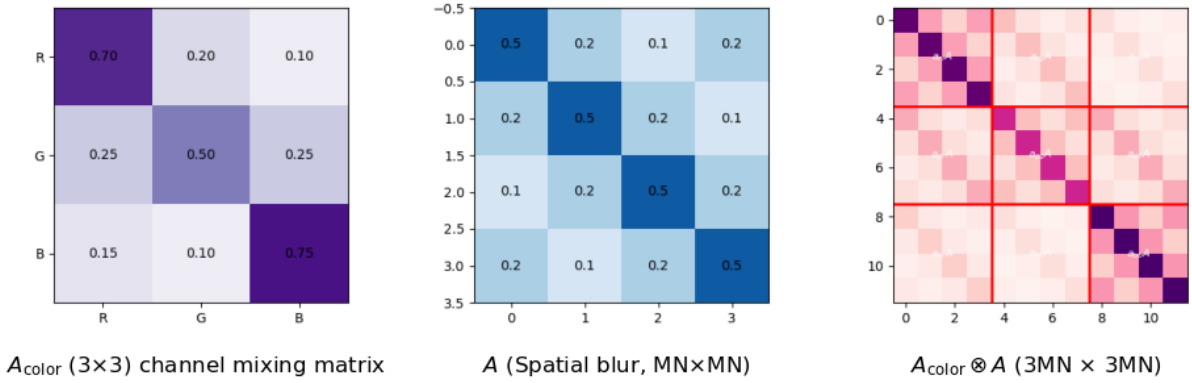


Figure 17: Visual example of kronecker product structure of the cross-channel blurring model

Here A_{color} is 3×3 (one entry per colour channel) and A is $MN \times MN$ (one entry per pixel), so $A_{\text{color}} \otimes A$ is a $3MN \times 3MN$ matrix. Applying this matrix to $\mathbf{x} = (\mathbf{x}_R, \mathbf{x}_G, \mathbf{x}_B)^T$ gives:

$$(A_{\text{color}} \otimes A)\,\mathbf{x} = \begin{pmatrix} a_{rr}A\mathbf{x}_R + a_{rg}A\mathbf{x}_G + a_{rb}A\mathbf{x}_B \\ a_{gr}A\mathbf{x}_R + a_{gg}A\mathbf{x}_G + a_{gb}A\mathbf{x}_B \\ a_{br}A\mathbf{x}_R + a_{bg}A\mathbf{x}_G + a_{bb}A\mathbf{x}_B \end{pmatrix},$$

which confirms that blurring happens in two steps:

1. Within-Channel blur: A is applied independently to each channel $(\mathbf{x}_R, \mathbf{x}_G, \mathbf{x}_B)$ via FFT.
2. Channel mixing: A_{color} linearly combines the three spatially-blurred channels.

Deblurring reverses these two steps in the same order: first invert the channel mixing by multiplying by A_{color}^{-1}, then apply Tikhonov regularization independently to each channel using the 2D FFT formula (14).

4.5 Examples

4.5.1 Example 1: Within-Channel Deblur

We apply Gaussian blur with $\sigma = 3.0$ independently to each RGB channel with Gaussian noise $\sigma_e = 0.02$. The regularization parameters for TSVD and Tikhonov are selected from the combined L-curve across all three channels.

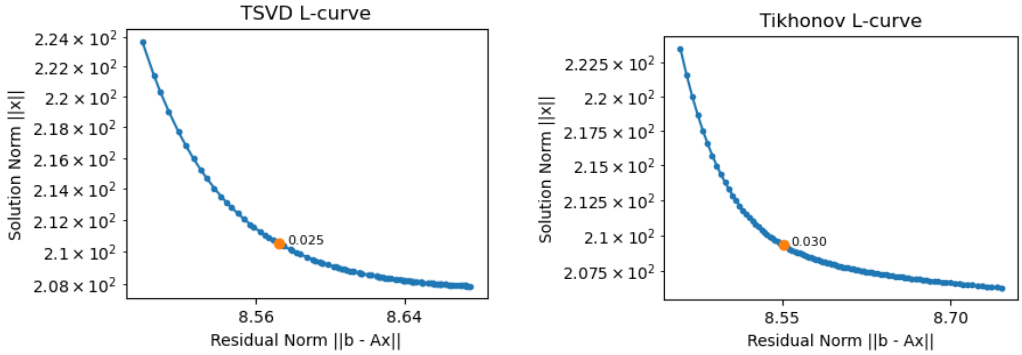


Figure 18: TSVD and Tikhonov L-curve for color blurred image



Figure 19: TSVD and Tikhonov results on color blurred image

Both TSVD and Tikhonov successfully recover the color image, with each channel deblurred independently using the same parameter to preserve color balance (within-channel deblurring).

4.5.2 Example 2: Cross-Channel Deblur

We use the mixing matrix from [1] Figure 7.3:

$$A_{\text{color}} = \begin{bmatrix} 0.70 & 0.20 & 0.10 \\ 0.25 & 0.50 & 0.25 \\ 0.15 & 0.10 & 0.75 \end{bmatrix}. \tag{18}$$

The same Gaussian PSF ($\sigma = 3.0$) is applied first for within-channel blurring, followed by cross-channel mixing via A_{color}, and Gaussian noise with $\sigma_e = 0.02$. Deblurring inverts A_{color}, then applies Tikhonov regularization per channel.

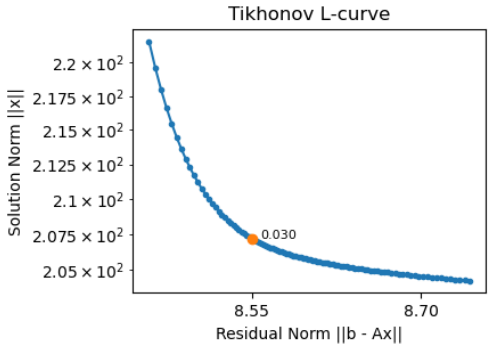


Figure 20: Tikhonov L-curve for cross-channel blur



Figure 21: Tikhonov result on cross-channel blurred image

4.6 Analysis and Discussion

Tikhonov and TSVD both recover recognizable images from blurred and noisy inputs, though perfect reconstruction is impossible once noise is present. With well-chosen parameters, both methods produce very similar results. The main difference appears at suboptimal (very large) values: TSVD can introduce ringing artifacts due to its hard cutoff, while Tikhonov degrades more smoothly through over-smoothing.

For within-channel deblurring, we use the same regularization parameter across all three RGB channels.

Since each channel has the same PSF and noise level, their L-curves are identical, and any channel can be used to find the best value. This allows us to use one shared parameter, which also helps preserve color balance in the reconstructed image.

The cross-channel deblurred image appears more colorful and grainy compared to the within-channel result, even with the same original image and regularization parameter. This happens because cross-channel blurring mixes information between RGB channels through A_{color}, so each observed channel contains contributions from the others rather than being independent. When inverting A_{color} to deblur, this mixing also spreads the noise across all three channels, amplifying it and leading to a more textured and saturated reconstruction.

Compared to motion blur, the optimal α values here are smaller. This is because the Gaussian PSF has a smoother spectrum, so less regularization is needed. Additionally, due to a consistent blur and noise σ, the optimal α values are relatively consistent through examples, with only slight differences in the L-curves. Together, these results demonstrate that independent per-channel regularization is effective when channels are decoupled, but cross-channel interactions introduce additional noise sensitivity.

5 3D Volumetric Blurring and Deblurring

In previous blurring and deblurring models, we have developed methods for 2D images, in this section we are going to extend these models for 3D images. For our data in this section, rather than having a pixel for a 2D image, we are going to have a voxel, which is a volumetric pixel, for a 3D image. Our example data is an MRI scan which produces a stack of 2D slices that together form a 3D volume, and blur occurs in all three spatial directions simultaneously.

5.1 The Dataset

We use MATLAB's built-in MRI dataset[3], which contains 27 axial brain slices of size 128×128. The full volume is a discrete array $X \in \mathbb{R}^{128\times128\times27}$. Each entry $X_{i,j,k}$ represents the intensity at spatial position (i, j, k), where i and j index the rows and columns of a single brain slice, and k indexes the slice number along the head-to-toe axis.

The raw data has integer pixel values in the range $[0, 88]$. We normalize by dividing by the maximum value so that all intensities lie in $[0, 1]$:

$$X_{i,j,k} \leftarrow \frac{X_{i,j,k}}{\max_{i,j,k} X_{i,j,k}}. \tag{19}$$

X denotes the true image that we want to recover.
This normalization ensures numerical stability during the Fourier-based computations.

5.2 3D Blurring

First, we are going to blur the 3D image dataset by constructing a 3D Gaussian PSF, applying the linear blurring model via FFT-based convolution, and adding Gaussian noise to simulate realistic measurement conditions.

5.2.1 3D Gaussian Point Spread Function (PSF)

The 2D Gaussian PSF was introduced in 2.2.1, Now we are going to extend this for 3D volumetric images.

The 3D Gaussian function

For an N-dimensional isotropic Gaussian, the general formula is [2]:

$$G_{N\mathrm{D}}(\vec{x};\sigma) = \frac{1}{(\sqrt{2\pi}\,\sigma)^N}\,\exp\!\left(-\frac{\|\vec{x}\|^2}{2\sigma^2}\right). \tag{20}$$

Setting $N = 3$ and writing $\vec{x} = (x, y, z)$ gives the 3D isotropic Gaussian PSF:

$$G(x, y, z) = \frac{1}{(2\pi)^{3/2}\,\sigma^3}\exp\!\left(-\frac{x^2+y^2+z^2}{2\sigma^2}\right). \tag{21}$$

The parameter σ plays the same role as in the 1D and 2D cases where it controls the width of the kernel in all three spatial directions equally, since the blur is isotropic. A larger σ spreads the blur over more voxels, while a smaller σ keeps the blur concentrated near the center.

Discretization and Normalization of 3D Gaussian Kernel

We evaluate Equation (21) on a discrete grid of integers centered at zero. The kernel size is chosen as $2\lceil 2\sigma\rceil + 1$ per axis, which is the convention used in MATLAB's `imgaussfilt3` function[3]. With using this, we can capture voxels within two standard deviations of the center and discard the negligibly small values beyond that range.

With $\sigma = 2$, the PSF is a $9 \times 9 \times 9$ array. After evaluation, we normalize so that all entries sum to 1:

$$G_{\text{norm}}[i,j,k] = \frac{G[i,j,k]}{\sum_{i,j,k} G[i,j,k]}. \tag{22}$$

This normalization ensures that blurring preserves the total intensity of the volume. This also makes PSF act like a weighted average rather than amplifying or reducing intensity. Since we normalize explicitly, the constant prefactor $(2\pi)^{-3/2}\sigma^{-3}$ in Equation (21) does not need to be calculated and it cancels after we normalize.

5.2.2 The Convolution Operation for 3D

We are going to use the 2D Convolution Operation from section 2.2 and extend it for 3D.

The matrix A encodes 3D convolution with the PSF. For each voxel (i,j,k) in the output, the blurred value is a weighted average of surrounding voxels in the input [1]:

$$B_{i,j,k} = \sum_p \sum_q \sum_r G_{p,q,r} \cdot X_{i-p,\, j-q,\, k-r}, \tag{23}$$

where $G_{p,q,r}$ is the entry in the 3D Gaussian PSF at position (p,q,r) which is the weight function. This ensures that the voxels closer to the center contribute more, while distant voxels contribute less. and $X_{i-p,\, j-q,\, k-r}$ is the image voxel at offset (p,q,r) away from position (i,j,k).

In vector form, if we reshape the 3D volume into a column vector $x = \text{vec}(X) \in \mathbb{R}^N$, the convolution becomes a matrix-vector product $b_{\text{exact}} = Ax$, where $A \in \mathbb{R}^{N \times N}$ with $N = 128 \times 128 \times 27 = 442{,}368$. Each row of A encodes the PSF weights for one output voxel. Constructing this matrix A explicitly and storing it is not efficient since it is a 442,368 $\times$ 442,368 matrix. Thus, we are going to use FFT to compute Ax implicitly.

5.2.3 The Fourier Transform for Convolution

Constructing A explicitly is not feasible since it is a huge matrix. Under periodic boundary conditions, the BCCB diagonalization from Section 2.2.4 extends to 3D, so $A = F^*\Lambda F$ where F is now the 3D DFT matrix. This reduces the blurring operation to element-wise multiplication in the frequency domain at a cost of $O(N \log N)$ instead of $O(N^2)$.

The 3D FFT extension is the same as the 2D case from Section 2.2.4. The blurring of 3D image then reduces to three steps[1]:

1. $\hat{x} = Fx$ (3D FFT of the image),
2. $\hat{b} = S\hat{x}$ (element-wise multiplication, multiply with diagonal matrix),
3. $b = F^*\hat{b}$ (3D inverse FFT to get blurred image),

In our implementation, `fftconvolve` from `scipy.signal` performs the full FFT-based convolution in a single call with `mode='same'` to preserve the input shape. In the deblurring step, we use `fftn` and `ifftn` directly rather than `fftconvolve`, since the Tikhonov formula (25) requires access to the individual spectral values λ_i and $\hat{b}_i$.

5.2.4 Adding noise

After convolution, we add Gaussian white noise to model measurement imperfections:

$$B_{\text{observed}} = (G * X) + e, \qquad e_{i,j,k} \sim \mathcal{N}(0,\, \sigma_e^2), \tag{24}$$

where σ_e controls the noise level.

5.2.5 Results

To visualize the effect of blurring on the 3D volume, we extract the middle axial slice ($k = 13$) and compare the results under different noise levels and PSF widths.

3D Blur With Different Noise Values

To Compare different noise value outcomes, we fix PSF width to $\sigma_{\text{blur}} = 2.0$. We blur the images with three different noise levels as $\sigma_e \in \{0.01, 0.02, 0.04\}$.

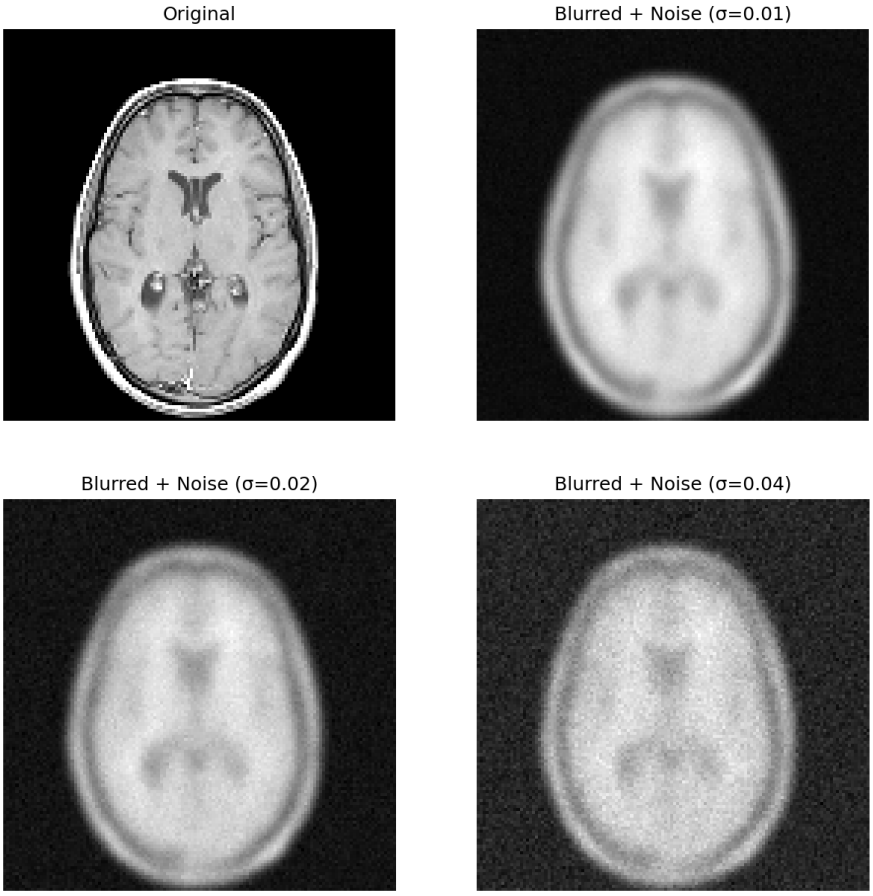


Figure 22: 3D Blur With Different Noise Values

As seen the Figure 22, increasing the noise values makes the images grainier and pixilated and makes deblurring and recovering the images harder.

3D Blur with Different Blur Widths (PSF)

To compare different PSF Widths, we fix the noise level as $\sigma_e = 0.02$. We blur the images with three different blur widths as $\sigma_{\text{blur}} \in \{1.0, 2.0, 4.0\}$.

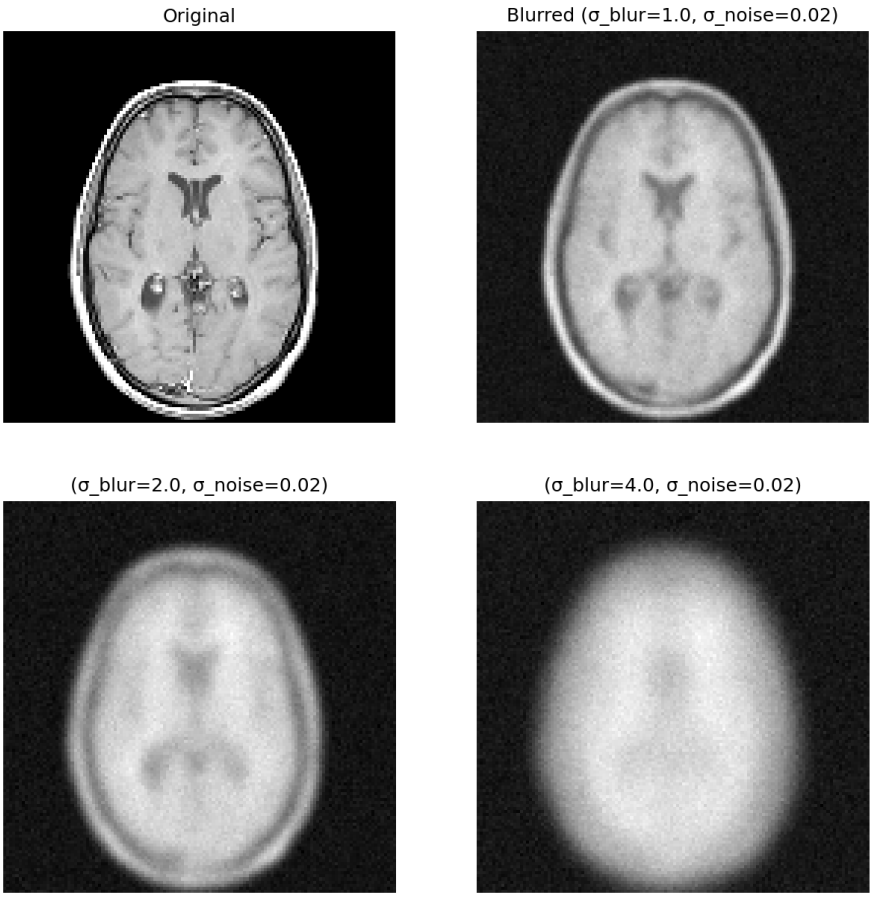


Figure 23: 3D Blur with Different Blur Widths (PSF)

As seen in Figure 23, as the PSF width gets larger, the blurred volume gets bigger, since the effect of the blur spreads further in the neighboring voxels. Larger PSF width makes the recovery of the details in the image harder.

5.3 3D Deblurring with Tikhonov Regularization

In this section, we attempt to recover 3D volumetric images that were blurred by using Tikhonov regularization in the frequency domain, and for efficient computation we are going to use the 3D FFT.

5.3.1 The Tikhonov Regularization Method

We use the Tikhonov Regularization mentioned in Section 2.4 to deblur the 3D images. In this section we are going to use Tikhonov to efficiently deblur 3D images which contain larger number of voxels compared to 2D images. We are going to use FFT to implement our 3D Volumetric Deblurring function.

5.3.2 Implementation Using 3D FFT

The BCCB diagonalization from Section 2.2.4 extends directly to 3D by replacing `fft2`/`ifft2` with `fftn`/`ifftn`. Substituting the 3D decomposition $A = F^*\Lambda F$ into the Tikhonov closed-form solution from Section 2.4.2 gives the same element-wise formula in the Fourier domain[1]:

$$\hat{x}_i = \frac{\overline{\lambda}_i \cdot \hat{b}_i}{|\lambda_i|^2 + \alpha^2}, \qquad i = 1, \ldots, N, \tag{25}$$

where λ_i are now obtained from the 3D FFT of the PSF and $\hat{b}_i$ from the 3D FFT of the blurred volume. The restored volume is $x_\alpha = \text{IFFT}_{3D}(\hat{x})$, requiring no explicit matrix construction or storage.

5.3.3 Choosing α

The regularization parameter α balances both errors we get in this equation[1]:

- Perturbation error: Which is the noise amplification. Noise amplification decreases as α increases, because larger α suppresses more high-frequency components.
- Regularization error: Which is the over-smoothing. Smoothing increases as α increases, because larger α also suppresses some real image data.

The optimal α minimizes their weighted sum.

In the code, we have the real data of the unblurred image, thus we are going to use the error based on the true values of the data. We pick the smallest relative error between the true data and the Tikhonov computed data: $\|x_{\text{restored}} - x_{\text{true}}\|/\|x_{\text{true}}\|$.

5.3.4 Results

Finding the Best Alpha Value

To find the best α value for our MRI dataset, we evaluate 30 values of α in logarithmic scale between $[10^{-3}, 10^{0}]$. After computing the relative error of each alpha, we choose the best resulting alpha with smallest relative error.

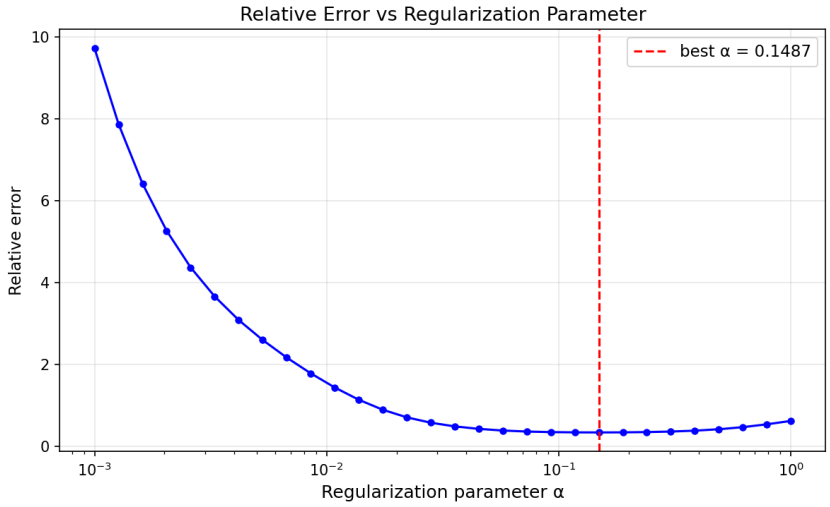


Figure 24: Finding the Best Alpha value based on relative error

Tikhonov Deblurring with Different α Values

To show how different α values effect the 3D volumetric deblurring, we compare Tikhonov deblurring on middle layer, $k = 13$ with small, best and large α values which are $\alpha = \{0.05, 0.1487, 0.5\}$.

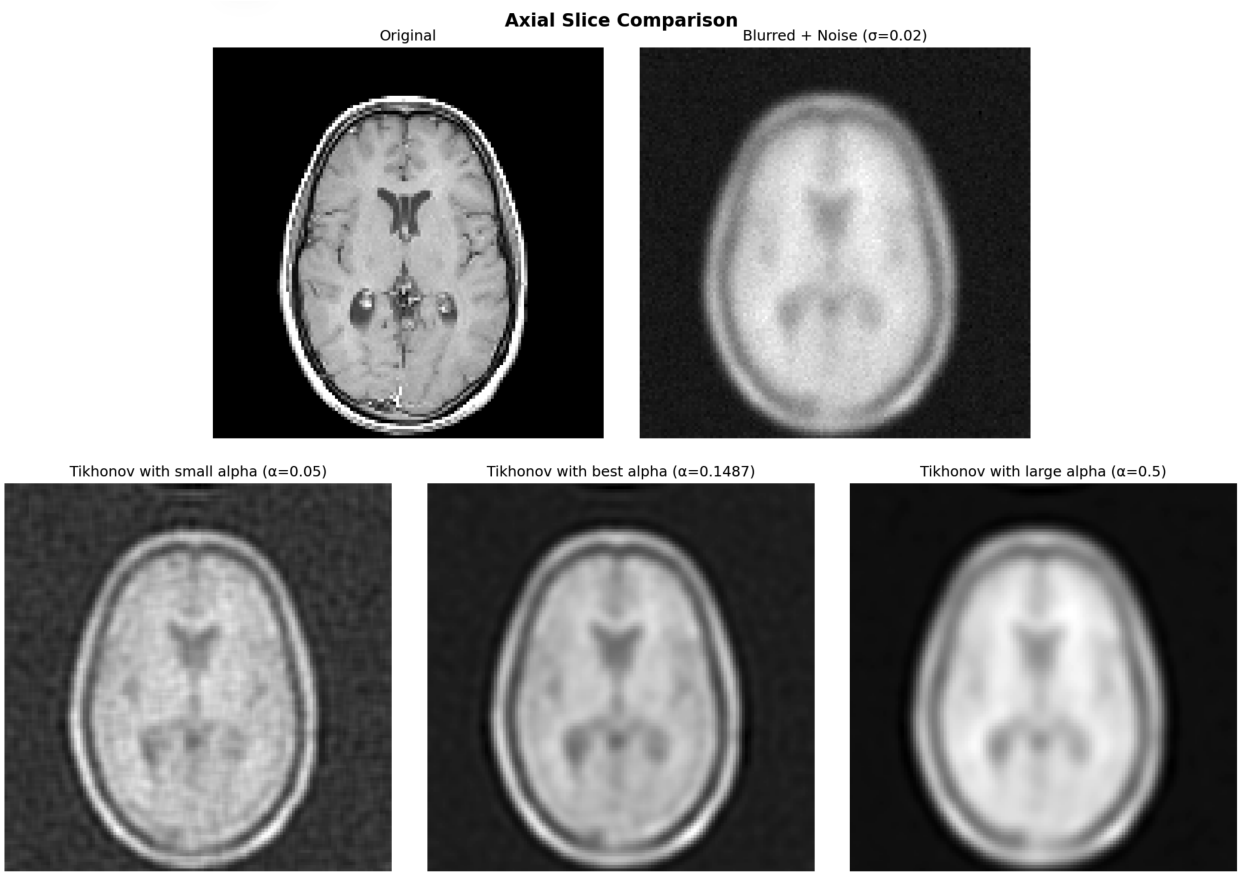


Figure 25: Axial comparisons of original, 3D blurred and Tikhonov Deblurred images with different alpha values

From Figure 25, when $\alpha = 0.05$, the result is sharper but contains visible noise artifacts since many high-frequency noise components pass the filter. When $\alpha = 0.5$, there is no visible noise but the image remains noticeably blurry, especially in the detailed parts. The best value for α recovers the most structural detail and also keeps the noise in an acceptable level.

All Axial Slices Original, Blurred and Tikhonov Deblurred

Following figures 26, 27, and 28 shows all 27 layers of the MRI image in the original state, blurred, and deblurred versions with optimal parameters.

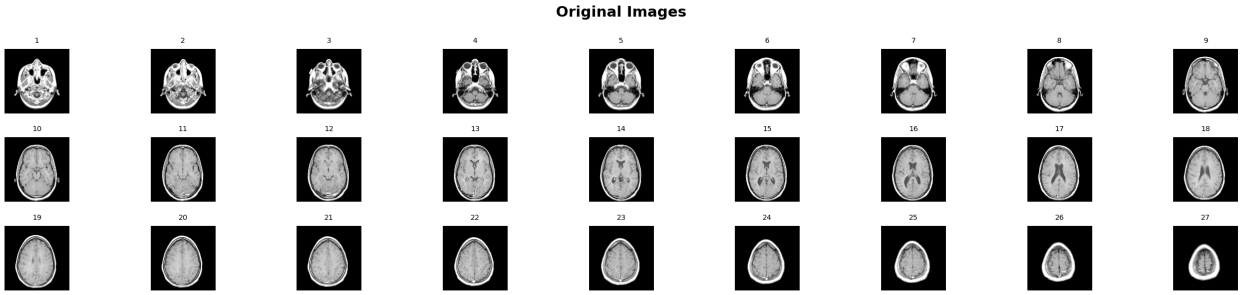


Figure 26: All Axial Slices of Original MRI Data

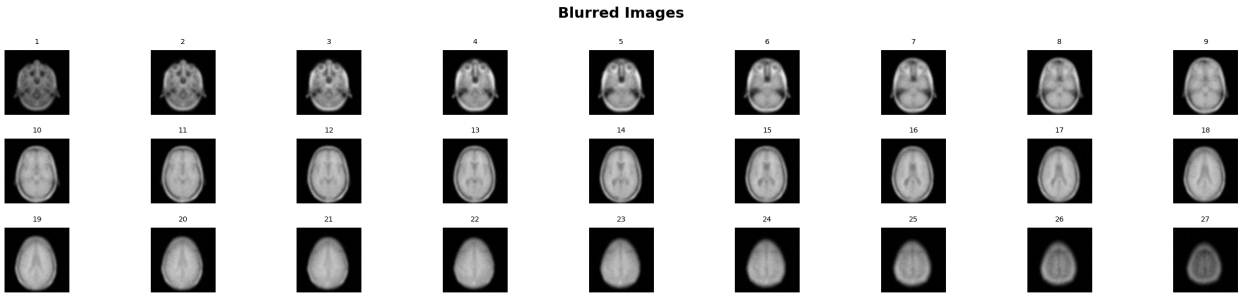


Figure 27: All Axial Slices of MRI Data after 3D Blurring

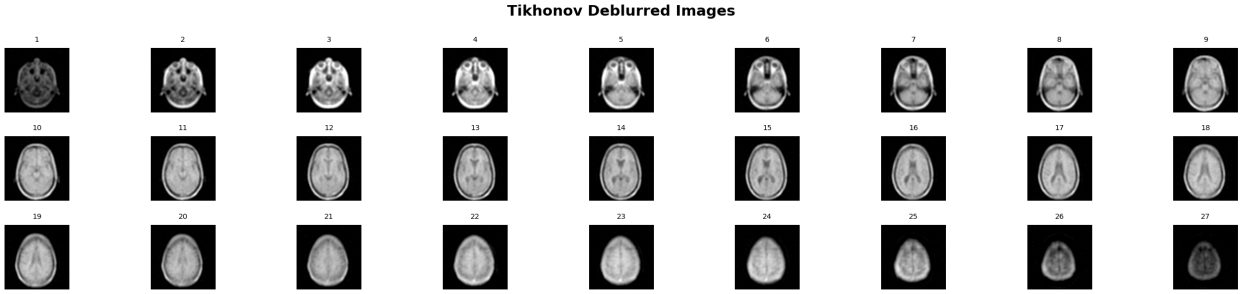


Figure 28: All Axial Slices of MRI Data after Tikhonov 3D Deblurring

From these figures, we can see that the Tikhonov deblurring can restore all 27 slices and not just the middle slice with optimal parameters. The deblurred images when compared to original images contain the details for edge and contrast which were lost after blurring.

5.3.5 Analysis and Discussion

The resulting deblurred slices of 3D images were recovered with optimized parameters, which gave the $\alpha = 0.1487$ from the relative error analysis. Most of the edges and details that were lost after blurring were recovered with Tikhonov Regularization Method using 3D FFT. Finding the optimal value for parameters is crucial to find the balanced output of noise level and over-smoothing since 3D volumetric images are being used in medical field like an MRI scan of the brain, being able to recover small details is an important factor.

When blurring images, varying noise levels and PSF widths showed that both factors degrade the quality of deblurring. Higher noise forces a larger α which sacrifices detail to suppress artifacts, while a wider PSF destroys more high-frequency content during convolution, which leaves less information to recover.
The results of 3D volumetric deblurring confirmed that the choice of α directly controls the trade-off between noise suppression and detail recovery, where too small α leaves visible noise and too large α over-smooths the image. The FFT-based approach was essential for making all computations feasible by reducing the cost from $O(N^2)$ to $O(N \log N)$ through the BCCB diagonalization under periodic boundary conditions.

6 Conclusion

In this project, we formulated image deblurring as a convex optimization problem and developed solutions using Tikhonov regularization across three different blurring methods; linear motion blur on grayscale images, Gaussian blur on color images, and 3D volumetric Gaussian blur on layered images like MRI data. In all three models, we could not deblur the images by the naive solution because of the ill-conditioned nature of the blurring matrix, since the blurring matrix amplifies noise by division of near-zero eigenvalues. We have used Tikhonov regularization to avoid this problem by suppressing the small eigenvalues while also preserving

meaningful image content. The choice of parameter α for deblurring balanced the noise amplification and over-smoothing of the resulting images. From the resulting images we have seen that how the method can be sensitive to parameters from estimating the PSF parameters.

7 Limitations

Our most significant limitation is that our approach assumes periodic boundary conditions. This assumption enables efficient FFT-based computation through BCCB diagonalization, but it also assumes that image continues in all direction by repeating itself. This can cause wrap-around artifacts near the edges where intensity seeps from one edge to the opposite edge.

Another limitation is the required knowledge of the PSF. In our models, we construct the PSF ourselves and use the same in deblurring directly but in real world applications, the PSF is unknown. The examples of real motion blurred photographs showed that the quality of deblurring is sensitive to the estimated blur length r and angle θ. And even small errors in these estimates can leave visible residual blur or introduce new artifacts.

Another limitation we face when deblurring images is selecting the regularization parameter α. The L-curve method used throughout this project is a geometric heuristic that does not always identify the optimal value, and its results can be sensitive to the range of parameter values tested. In the MRI example, we have used relative error to get the optimal values for α and this cannot be done if we do not have the original images.

8 Future Work

Natural future extensions would be to address our limitations. One direction is to explore non-periodic boundary conditions. Using alternative boundary models may produce more realistic results.

Another direction is to improve PSF estimation for real-world images. Developing more systematic or automated methods for estimating blur parameters could improve reconstruction quality.

For colored images, more complex cross-channel models could be considered. The current approach uses a fixed mixing matrix, but real imaging systems likely involves more complicated interactions between channels, which affects deblurring performance.

More advanced regularization methods could be explored. While Tikhonov regularization provides stable results, methods such as total variation may better preserve edges and fine details while still controlling noise.

9 Code

GitHub Repository: `https://github.com/luna-y-kim/ImageDeblurring`

References

[1] Per C. Hansen, James G. Nagy, and Dianne P. O'Leary. *Deblurring images: matrices, spectra, and filtering*. SIAM, Society for Industrial and Applied Mathematics, Philadelphia, 2006.

[2] Bart M. Ter Haar Romeny. The Gaussian kernel. In *Front-End Vision and Multi-Scale Image Analysis*, chapter 3. Springer, 2003. Available at `https://pages.stat.wisc.edu/~mchung/teaching/MIA/reading/diffusion.gaussian.kernel.pdf.pdf`.

[3] The MathWorks, Inc. `imgaussfilt3`: 3-D Gaussian filtering of 3-D images. MATLAB Documentation, Image Processing Toolbox, 2024. Accessed: 2026-04-09.